Tarea 5

Guía de Laboratorio para probar el programa sniffer

De manera previa a la realización de esta tarea, les recomiendo ver el PDF de la teoría así como los comentarios en el código del sniffer.c:

PDF

https://cromanzamitiz.github.io/arquitectura-cliente-servidor/tema3/Clase_Sniffer_Redes_y_Cliente_Servidor.pdf

sniffer.c

https://cromanzamitiz.github.io/arquitectura-cliente-servidor/tema3/sniffer.c

Demostración práctica utilizando sniffer.c, sigan esta secuencia en tres terminales de Linux:

1. **Terminal 1 (Compilación y activación del Sniffer):**

> IMPORTANTE: si lo realizan en máquinas diferentes, la que debe activar el modo promiscuo es la máquina del sniffer. Ni la del servidor ni la del cliente. ¿Por qué? El sniffer es un observador pasivo. Para que su tarjeta de red capture las tramas que viajan por el medio físico de la red virtual (por ejemplo, el switch virtual de VirtualBox o VMware) aunque vayan dirigidas a otra computadora, necesariamente debe estar en modo promiscuo. Si no lo activa, su tarjeta solo capturará el tráfico propio de esa máquina virtual.

```
# Esto se debe realizar como root
# Activar modo promiscuo (ajustar interfaz o usar lo)
# reemplacen eth0 por el nombre de la interface de red o por lo si usan loopback
ip link set eth0 promisc on

gcc sniffer.c -o sniffer
./sniffer
```

Arquitectura Cliente Servidor

Problemas Salida Consola de depuración Terminal Puertos

```
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $ sudo su
root →/workspaces/arquitectura-cliente-servidor (main) $ ip link set lo promisc on
root →/workspaces/arquitectura-cliente-servidor (main) $ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:00:d8:76:dc  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.3.247  netmask 255.255.0.0  broadcast 10.0.255.255
        inet6 fe80::e6fa:deff:fe1a:c523  prefixlen 64  scopeid 0x20<link>
        ether e4:fa:de:1a:c5:23  txqueuelen 1000  (Ethernet)
        RX packets 8956675  bytes 13034856946 (13.0 GB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 261456  bytes 72310061 (72.3 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=329<UP,LOOPBACK,RUNNING,PROMISC>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 13484  bytes 41174506 (41.1 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 13484  bytes 41174506 (41.1 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root →/workspaces/arquitectura-cliente-servidor (main) $
```

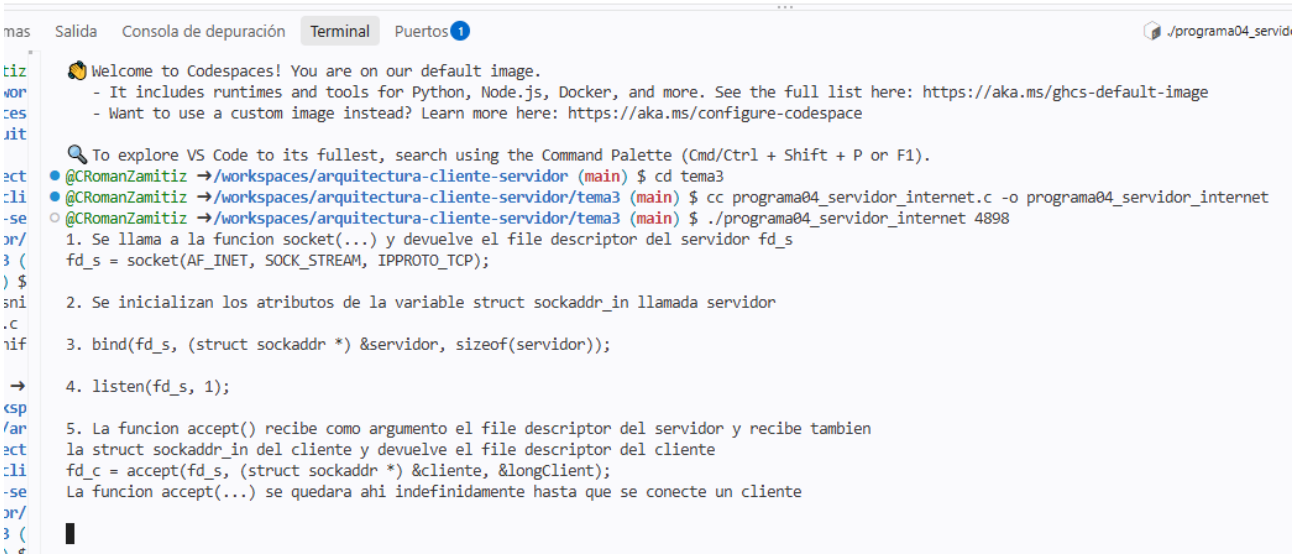
Con el commando ifconfig comprueban que su interface de red ya está en modo promiscuo.
Posteriormente, compilan y levantan el sniffer:

```
root →/workspaces/arquitectura-cliente-servidor (main) $ cd tema3
root →/workspaces/arquitectura-cliente-servidor/tema3 (main) $ cc sniffer.c -o sniffer
root →/workspaces/arquitectura-cliente-servidor/tema3 (main) $ ./sniffer
>>> SNIFFER ACTIVO: Escuchando tramas Ethernet en la red...
>>> Esperando trafico TCP en el puerto 4898...
```

2. Terminal 2 (Levantar un Servidor. Puede ser el servidor del programa 04 o 05 en el puerto 4898:

```
cc programa04_servidor_internet.c -o programa04_servidor_internet
./programa04_servidor_internet 4898
```

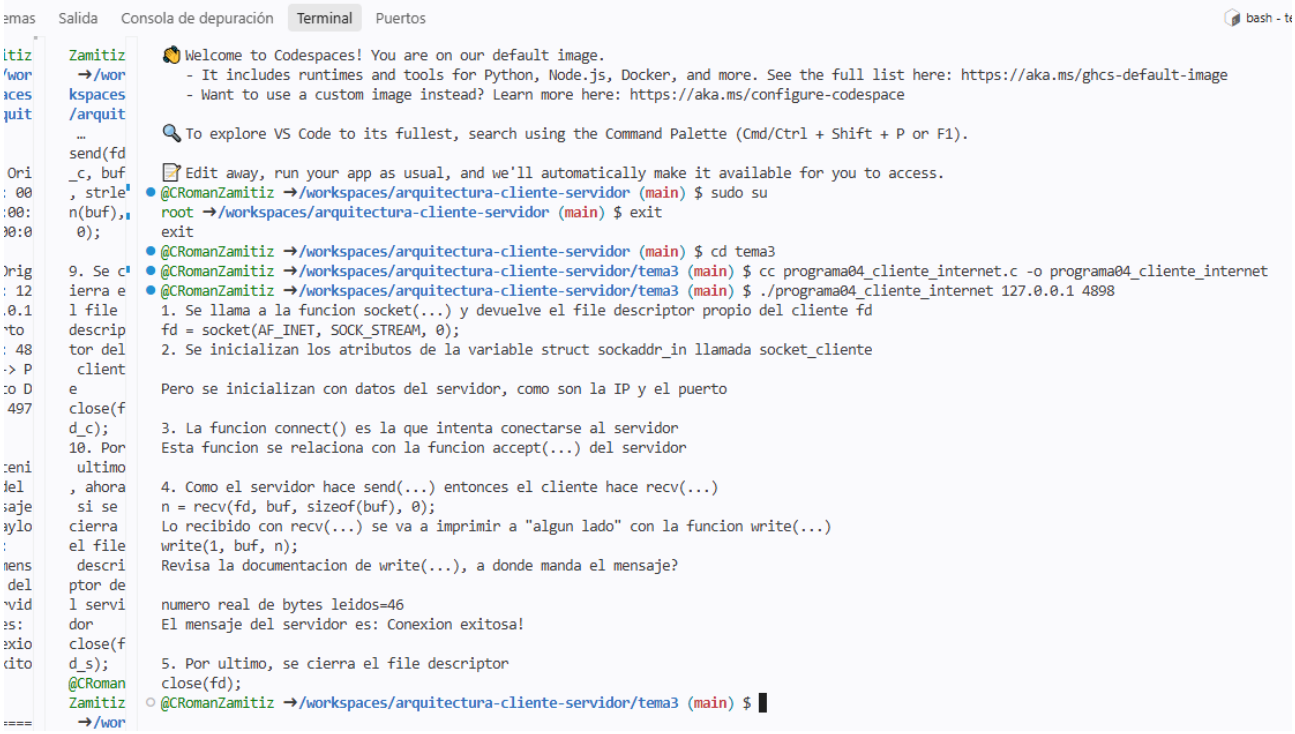
Arquitectura Cliente Servidor



3. Terminal 3 (Levantar el Cliente que recibe datos en texto plano):

```
cc programa04_cliente_internet.c -o programa04_cliente_internet
./programa04_cliente_internet 127.0.0.1 4898
```

Arquitectura Cliente Servidor



Al presionar Enter, observarán en tiempo real cómo la Terminal 1 intercepta la trama Ethernet, desencapsula IP y TCP, e imprime el payload en texto plano.

```
root ➜/workspaces/arquitectura-cliente-servidor (main) $ cd tema3
root ➜/workspaces/arquitectura-cliente-servidor/tema3 (main) $ cc sniffer.c -o sniffer
root ➜/workspaces/arquitectura-cliente-servidor/tema3 (main) $ ./sniffer
>>> SNIFFER ACTIVO: Escuchando tramas Ethernet en la red...
>>> Esperando trafico TCP en el puerto 4898...

==========================================================
---[ PAQUETE INTERCEPTADO EN TEXTO PLANO ]---
MAC Origen: 00:00:00:00:00:00
IP Origen : 127.0.0.1
Puerto Src: 4898 -> Puerto Dst: 49786

Contenido del mensaje (Payload):
El mensaje del servidor es: Conexion exitosa!

==========================================================

==========================================================
---[ PAQUETE INTERCEPTADO EN TEXTO PLANO ]---
MAC Origen: 00:00:00:00:00:00
IP Origen : 127.0.0.1
Puerto Src: 4898 -> Puerto Dst: 49786

Contenido del mensaje (Payload):
El mensaje del servidor es: Conexion exitosa!

==========================================================
```

Anexo: Código Fuente de la Práctica (sniffer.c)

```c
/*
 * ============================================================================
 * Práctica de Laboratorio: Sniffer de Red en Capa 2 (Raw Sockets)
 * Descripción: Intercepta y analiza tramas Ethernet, paquetes IPv4 y
 * segmentos TCP directamente desde la tarjeta de red.
 * Advertencia: Este programa requiere privilegios de superusuario (root)
 * y la interfaz de red en "Modo Promiscuo" para ver tráfico ajeno.
 * ============================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/ip.h>       // Estructura de la cabecera IPv4 (struct iphdr)
#include <netinet/tcp.h>      // Estructura de la cabecera TCP (struct tcphdr)
#include <netinet/if_ether.h> // Estructura de la cabecera Ethernet (struct ethhdr)
#include <arpa/inet.h>        // Funciones de conversión como inet_ntoa y ntohs
#include <unistd.h>
```

```
#include <linux/if_packet.h>  // Definiciones para AF_PACKET (Capa de Enlace)
#include <net/if.h>

int main() {
    int raw_socket;
    // Reservamos 64KB de memoria. Este es el tamaño máximo teórico de un paquete IP.
    unsigned char *buffer = (unsigned char *)malloc(65536);

    /* * PASO 1: CREACIÓN DEL SOCKET EN CRUDO (RAW SOCKET)
     * AF_PACKET: Nos permite interactuar directamente con el driver del dispositivo
     * de red (Capa 2 del modelo OSI - Enlace de Datos).
     * SOCK_RAW:  Le decimos al kernel que no procese el paquete, lo queremos "crudo".
     * ETH_P_ALL: Queremos capturar TODOS los protocolos de Ethernet (IPv4, ARP, IPv6,
etc.).
     * Nota: Usamos htons() porque ETH_P_ALL debe enviarse en Network Byte Order.
     */
    raw_socket = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    if (raw_socket < 0) {
        perror("Error al crear el socket (¿Ejecutaste el programa con sudo?)");
        return 1;
    }

    printf(">>> SNIFFER ACTIVO: Escuchando tramas Ethernet en la red...\n");
    printf(">>> Esperando tráfico TCP en el puerto 4898...\n");

    // Bucle infinito para escuchar paquetes continuamente
    while (1) {
        /*
         * PASO 2: RECIBIR EL PAQUETE
         * recvfrom captura los bits que llegan a la antena/cable y los guarda en
'buffer'.
         * data_size nos dirá cuántos bytes exactos capturamos.
         */
        int data_size = recvfrom(raw_socket, buffer, 65536, 0, NULL, NULL);

        // Si hay un error de lectura (ruido, interferencia), ignoramos y continuamos
        if (data_size < 0) {
            continue;
        }

        /*
         * PASO 3: ANÁLISIS DE LA CAPA 2 (ENLACE DE DATOS - ETHERNET)
         * El buffer empieza con la cabecera Ethernet, que siempre mide 14 bytes.
         * Hacemos un "casting" para que C lea los primeros 14 bytes como una estructura
'ethhdr'.
         */
        struct ethhdr *eth = (struct ethhdr *)buffer;

        // Verificamos el campo 'h_proto' de la cabecera Ethernet.
        // ETH_P_IP (0x0800) significa que el paquete encapsulado dentro de Ethernet es
IPv4.
```

```
        // Usamos ntohs() porque el paquete viene de la red (Big-Endian) y nuestra CPU
        // necesita procesarlo en su formato nativo (Host Order / Little-Endian).
        if (ntohs(eth->h_proto) == ETH_P_IP) {

            /*
             * PASO 4: ANÁLISIS DE LA CAPA 3 (RED - IPv4)
             * Para leer la cabecera IP, debemos "saltarnos" la cabecera Ethernet.
             * Sumamos sizeof(struct ethhdr) (14 bytes) a la dirección base del buffer.
             */
            struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct ethhdr));

            // El campo 'protocol' dentro de IPv4 nos indica qué hay en la capa
superior.
            // El valor 6 es el estándar de la IANA para el protocolo TCP.
            if (iph->protocol == 6) {

                /*
                 * PASO 5: ANÁLISIS DE LA CAPA 4 (TRANSPORTE - TCP)
                 * Las cabeceras IP tienen un tamaño variable (usualmente 20 bytes).
                 * El campo 'ihl' (Internet Header Length) indica la longitud en
"palabras de 32 bits".
                 * Multiplicamos por 4 para obtener los bytes reales (ej. 5 * 4 = 20
bytes).
                 */
                unsigned short iphdrlen = iph->ihl * 4;

                // Saltamos la cabecera Ethernet (14) + la cabecera IP (ej. 20) para
llegar a TCP
                struct tcphdr *tcph = (struct tcphdr *)(buffer + sizeof(struct ethhdr) +
iphdrlen);

                // Filtramos para capturar SOLO la comunicación de nuestro laboratorio
(puerto 4898).
                // Revisamos tanto el puerto de destino (petición) como el origen
(respuesta).
                if (ntohs(tcph->dest) == 4898 || ntohs(tcph->source) == 4898) {

                    /*
                     * PASO 6: CÁLCULO DEL PAYLOAD (LOS DATOS REALES)
                     * Igual que IP, la cabecera TCP tiene tamaño variable.
                     * 'doff' (Data Offset) nos da el tamaño de la cabecera TCP en
palabras de 32 bits.
                     */
                    int tcp_header_len = tcph->doff * 4;

                    // Tamaño total de TODAS las cabeceras sumadas (Ethernet + IP + TCP)
                    int total_headers_size = sizeof(struct ethhdr) + iphdrlen +
tcp_header_len;

                    // El payload (el mensaje "Hola mundo" o el "GET / HTTP") empieza
                    // justo después de todas las cabeceras.
```

```
                    unsigned char *payload = buffer + total_headers_size;

                    // El tamaño de los datos es el paquete completo menos las cabeceras
                    int payload_size = data_size - total_headers_size;

                    // Si efectivamente hay datos (no es solo un paquete de control como
SYN o ACK)
                    if (payload_size > 0) {

printf("\n============================================================");
                        printf("\n---[ PAQUETE INTERCEPTADO EN TEXTO PLANO ]---");

                        // Imprimimos las direcciones MAC (Capa 2) extrayéndolas byte
por byte en Hexadecimal
                        printf("\nMAC Origen: %02x:%02x:%02x:%02x:%02x:%02x",
                               eth->h_source[0], eth->h_source[1], eth->h_source[2],
                               eth->h_source[3], eth->h_source[4], eth->h_source[5]);

                        // Imprimimos la IP de origen (Capa 3) convirtiéndola de binario
a texto legible
                        printf("\nIP Origen : %s", inet_ntoa(*(struct in_addr *)&iph-
>saddr));

                        // Imprimimos los puertos involucrados (Capa 4)
                        printf("\nPuerto Src: %d -> Puerto Dst: %d", ntohs(tcph-
>source), ntohs(tcph->dest));

                        // Imprimimos el payload (Capa de Aplicación). Usamos %.*s para
imprimir
                        // exactamente la cantidad de bytes que calculamos, evitando
leer memoria extra.
                        printf("\n\nContenido del mensaje (Payload):\n%.*s\n",
payload_size, payload);

printf("============================================================\n");
                    }
                }
            }
        }
    }

    // Liberar la memoria reservada antes de cerrar (buena práctica en C)
    free(buffer);
    close(raw_socket);
    return 0;
}
```

Se entrega captura de pantalla de las 3 terminales